Measure, Don't Predict: Accuracy-Guarded Latency Optimization of Object Detectors on a Client NPU

Julien Beloin
Independent researcher
https://github.com/Zeta-12/OptiNPU

***Abstract*—Laptops now ship with neural processing units (NPUs) that promise low-power inference, yet modern object detectors run poorly on them. On an Intel Core Ultra NPU, none of seven state-of-the-art real-time detectors runs fastest on the NPU alone, FLOPs do not predict latency, the vendor's per-operation counters overlap, and the compiler can silently cost up to 4.6 AP or hang the device. We present OptiNPU, an open-source tool that profiles, validates and optimizes a model for the NPU using on-device measurements only. It rests on an observation that we verify: latency does not depend on the values of dense weights, so any variant of a model, such as a bypassed region, a rewritten operation, or an architecture not yet retrained, can be timed with any weights, before it is trained or calibrated. OptiNPU (i) attributes latency by bypassing module regions in paired, randomized rounds, repeated until a 95 % confidence interval is narrow; (ii) guards accuracy at the task level, with detection agreement and a paired non-inferiority test on COCO AP, repeated on held-out images for approximations; and (iii) greedily applies exact and approximate rewrites, including a latency-guided mixed-precision search, and reports retraining rewrites with their measured latency. On five models, a repeated measurement always picks the best intervention, while acting on vendor counters, on an additive cost model, or on a regressor fitted to the other models' measurements loses on average 31 %, 60 % and 21 % of the best saving. The optimizer makes YOLO26-N 2.84× and YOLO26-S 2.02× faster on the NPU, repairing a silent 4.6-AP loss and a device hang, with AP within 0.1 points of CPU FP32. The held-out confirmation rejects a ResNet-50 mix that loses 0.64 points on the report images, but also three candidates whose mean loss stays within the budget: on the NPU, a single INT8 deployment remains (RF-DETR-N, 1.02×), and two DETR repairs cost latency. NNCF's accuracy-aware quantization, which validates on the CPU, accepts losses our test cannot certify, and two of its six models lose 1.5 and 18 points on the NPU. Optimized the same way, the iGPU is faster on seven of the ten models, but the NPU deployment uses 1.2 to 2.3× less energy per inference on nine of them.**

***Index Terms*—neural processing unit, object detection, latency profiling, quantization, mixed precision, compiler validation**

I. Introduction

Client processors now integrate a neural processing unit (NPU) next to the CPU and the integrated GPU (iGPU): Intel Core Ultra, AMD Ryzen AI, Qualcomm Snapdragon X. Their purpose is to run sustained inference at low power and to leave the CPU and GPU free for the user. Real-time object detection is a natural workload for them, and the detectors of the last two years, mostly descendants of DETR [1] (RT-DETR [2], [3], D-FINE [4], DEIM [5], [6], RF-DETR [7], YOLO26 [8]) reach 40 to 53 COCO AP [9] at a few GFLOPs.

Deploying these detectors on an NPU is nevertheless a trial-and-error exercise. Our measurements on an Intel NPU 3720 (Section II) show that:

- none of seven detectors is fastest on the NPU alone: they run 1.4 to 3.6× slower than on the iGPU or in a hybrid NPU+iGPU placement;
- FLOPs do not predict latency: YOLO26-N (5.4 GFLOPs) takes 26.6 ms, ResNet-50 at 224 px (8.2 GFLOPs) 3.9 ms;
- the vendor's per-operation counters overlap, so their sum is several times the real latency;
- the compiler fails silently: in its default two-tile mode, YOLO26-N loses 4.6 AP, RT-DETRv2 3.8 AP, and YOLO26-S hangs the device after a few images.

Existing tools predict rather than measure. Latency predictors learn per-kernel costs and add them up [10]–[12]. Hardware-aware search and compression use such predictors or lookup tables [13]–[16]. Mixed-precision methods model the latency of a configuration as the sum of per-layer costs [17]. On this NPU, none of these assumptions holds: costs are not additive, and the compiled graph may be wrong.

We take the opposite approach: *measure, don't predict*. One property of the NPU makes this affordable: **latency does not depend on the values of dense weights**. A variant of a model can therefore be timed with any weights, before it is trained, calibrated or even designed in PyTorch; only data-dependent operations and weight sparsity need care. We build OptiNPU on this property. Our contributions are:

1) a characterization of a client NPU that explains why prediction fails there: non-additive costs, overlapping counters, value-dependent operations, exploited weight sparsity, and silent compiler errors (Sections II and IV-A);
2) a measured profiler, based on region bypassing and paired, randomized, adaptive rounds, which picks better interventions than vendor counters, an additive cost model and a learned regressor on 150 interventions over five models (Section V-B);
3) a task-level accuracy guard that combines detection agreement and a paired non-inferiority test on AP, calibrated on real compiler failures, and confirms approximations on held-out images (Section IV-C);
4) a three-tier optimizer (exact, approximate, retraining), with a latency-guided mixed-precision search that does not assume additive costs, evaluated on seven detectors and three image classifiers, against NNCF's accuracy-

TABLE I
DETECTORS ON THE INTEL NPU 3720 (BATCH 1; AP ON THE FULL COCO VAL2017, HENCE HIGHER THAN ON THE REPORT IMAGES OF TABLES VII AND XI). OTHER PLACEMENTS RUN THE ORIGINAL FP16 IR. THE NPU-ONLY COLUMN GIVES THE FASTEST EXACT SETTING (ONE OR TWO TILES).

Model	AP	Fastest placement	NPU only (ms, AP)	Ratio
YOLO26-N	40.3	iGPU 10.9 ms	29.3 (1 tile, 40.2)	2.7×
DEIMv2-N	42.9	iGPU 15.1 ms	23.9 (2 tiles, 42.0)	1.6×
D-FINE-N	42.8	NPU+iGPU 15.5 ms	25.9 (2 tiles, 42.6)	1.7×
YOLO26-S	47.7	iGPU 23.8 ms	41.1 (1 tile, 47.7)	1.7×
RF-DETR-N	48.4	iGPU 23.0 ms	83.4 (2 tiles, 48.4)	3.6×
D-FINE-S	48.5	NPU+iGPU 34.0 ms	47.0 (2 tiles, 48.4)	1.4×
RT-DETRv2-R18	48.1	NPU+iGPU 34.9 ms	80.7 (1 tile, 48.0)	2.3×

aware quantization and against the iGPU optimized the same way (Sections IV-D and V-C to V-F);

5) an open-source implementation (Apache-2.0) at https://github.com/Zeta-12/OptiNPU, which runs every compilation and inference in a watched child process.

II. BACKGROUND AND MOTIVATION

A. The Intel NPU 3720

The NPU of the Intel Core Ultra 5 235U has two neural compute tiles. Each tile has DPU arrays, which run convolutions and matrix products, SHAVE DSP cores, which run the other operations, and DMA engines. OpenVINO [18] compiles a whole graph for it, given in its intermediate representation (IR: an XML graph and a binary file of weights), in FP16 or INT8, and the `NPU_TILES` option selects one or two tiles (two by default). Dense CNNs at 640 px (ResNet-18 to -50) reach 2.3 to 3.3 effective TOPS in FP16 and 4.8 to 6.7 in INT8, counting two operations per multiply-accumulate, for a nominal peak of 6.1 and 12.3. The NPU only runs inference.

B. Today's detectors on this NPU

We exported seven detectors to OpenVINO and evaluated them on the full COCO val2017 on the CPU, the iGPU and the NPU (one and two tiles), in FP16 and INT8. Table I summarizes the results; our harness reproduces, within 0.2 points, the AP published with each checkpoint (the end-to-end AP for YOLO26; for RF-DETR-N, the repository's 48.4 rather than the paper's 48.0). Six observations motivate this work.

O1: the NPU alone is never the fastest placement. The NPU-only latency is 1.4 to 3.6× that of the best placement (Table I).

O2: FLOPs do not predict latency. YOLO26-N (5.4 GFLOPs) takes 26.6 ms on two tiles and 29.3 on one, while ResNet-50 at 224 px (8.2 GFLOPs, counted as Ultralytics does, two per multiply-accumulate) takes 3.9 ms. Over 20 backbone configurations, FLOPs rank latency only loosely (Spearman's $\rho = 0.70$): PlainNet-A0 (22 GFLOPs) runs in 8.3 ms, EfficientNet-B0 (6.3 GFLOPs) in 18.7 ms. Micro-benchmarks of 396 blocks show where the time goes: depthwise convolutions reach only 0.1 to 0.3 TOPS; SiLU and HardSwish add 30 to 55 % to a layer; a LayerNorm over 6 400 tokens costs almost as much as an MLP; a `TopK` inside the graph costs 1.2 to 2.5 ms, against 0.13 ms on the host CPU. The latency of a detector depends on which operations it uses, not on how many FLOPs it has.

O3: vendor counters overlap. The NPU reports per-task durations. For YOLO26-N (26 ms on two tiles), the SHAVE tasks add up to 45.9 ms and the DPU tasks to 18.9 ms. On four models, the counters sum to 123 to 297 ms for inferences of 17 to 31 ms. The same model gave a sum of 66 ms in one run and 126 ms in the next.

O4: costs are not additive. Timing truncated prefixes of a graph gives some segments negative costs, down to -3.7 ms, because the compiler makes different decisions on each truncated graph.

O5: the compiler fails silently. In two-tile mode, YOLO26-N loses 4.6 AP, RT-DETRv2 3.8 and DEIMv2-N 0.9, without any error. YOLO26-S loses the device (`ZE_RESULT_ERROR_DEVICE_LOST`) after 4 to 7 different images; one repeated input passes. Every operation is correct when compiled alone, attention and `TopK` included. The faults come from decisions the compiler makes on the whole graph, so correctness must be checked on the deployed network with real inputs.

O6: INT8 is not a universal lever. INT8 makes ResNet-50 at 640 px 2.4× faster. It makes D-FINE-N slower (61.6 ms against 43.1 on one tile) and gives YOLO26-S only 13 %, for a loss of 0.3 to 2.6 AP.

O1–O6 rule out a static cost model: a tool for this NPU must measure latency and must check correctness.

III. RELATED WORK

Latency prediction. nn-Meter [10] detects kernel fusions and sums kernel-level predictions; on an Intel VPU, only 83 % of its predictions fall within 10 % of the measured latency. HW-NAS-Bench [11] and BRP-NAS [12] learn latency predictors for architecture search. O3 and O4 show that kernel costs do not add up on our NPU; Section V-B uses such an additive model as a baseline.

Hardware-aware design and compression. NetAdapt [13] adapts networks with empirical latency measurements; ProxylessNAS [14] and MnasNet [15] put latency into the search objective; AMC [16] prunes under a resource budget. MobileDets [19] and accelerator-aware NAS [20] show that edge accelerators favor regular, full convolutions over depthwise ones, which O2 confirms. These works produce new architectures and require training. OptiNPU starts from a given model and first applies the changes that need no training.

Mixed-precision quantization. HAQ [21] learns per-layer bit widths with reinforcement learning and hardware feedback. HAWQ [22] ranks layers by Hessian sensitivity, and HAWQ-V3 [17] picks bit widths under a latency budget expressed as a sum of per-layer latencies. On the NPU we find this sum misleading (Section V-I2). NNCF's accuracy-aware quantization [23] returns the most harmful quantizers to float until a validation drop is met, ignoring latency and validating on the CPU (Section V-E). OptiNPU builds on NNCF post-training

quantization and measures every candidate mix as a whole, on the device.

Compilers and graph rewriting. TVM [24] and Ansor [25] tune kernels; TASO [26] and PET [27] search graph substitutions, the latter including partially equivalent ones with corrections. Vendor NPU compilers are closed, so OptiNPU works at the level of the model graph and the compile options. NNSmith [28] finds compiler bugs with generated graphs. We instead check the accuracy of the user's own deployed model on its task.

Heterogeneous execution and on-device studies. CoDL [29] and Band [30] split inference across mobile processors. We keep the network on the NPU, and use other devices only as a reference point. AI Benchmark [31], Almeida et al. [32] and MLPerf [33] characterize on-device inference. For measurement methodology we follow Hoefler and Belli [34] and randomize the execution order to avoid the biases described by Mytkowicz et al. [35].

IV. OptiNPU

OptiNPU takes a model (OpenVINO IR, ONNX or PyTorch with an example input), a *task* that turns images into inputs and outputs into predictions, a few guard images, and optionally an annotated evaluation set. It returns a measured profile, an optimized deployment (one or two IR parts with their devices and compile options), and a report (Fig. 1).

A. Premise: latency does not depend on dense weight values

We time each model with its trained weights and with variants of its inputs and weights, in paired adaptive rounds (Table II). Without an in-graph selection, real images, all-zero inputs and random dense weights change latency by at most 1.4 %. Two exceptions matter:

- **Data-dependent operations.** YOLO26 selects boxes with an in-graph `TopK`: on real images and with random weights, YOLO26-N runs 4.6 to 5.6 % faster than on the fixed noise input. Without this selection, which OptiNPU moves to the CPU, the premise holds again. D-FINE samples deformable attention [36] at data-dependent locations, with little effect (0.7 % at most).
- **Weight sparsity.** The NPU skips zero weights: ResNet-50 is 26 and 35 % faster with 50 and 90 % of its weights at zero, and its mixed-INT8 deployment 24 and 39 %. Only 1.6 to 3.6 % of the INT8 weights of our deployments are zero, and setting them to one changes nothing measurable. Zero activations are not skipped: with dense non-positive weights and negative biases, every ReLU outputs zero, yet neither ResNet-50 ($+0.9 \pm 0.5$ %) nor YOLO26-N with ReLU (-0.2 ± 1.9 %) gets faster.

This premise has two consequences. OptiNPU can time a rewrite before training it, for example SiLU replaced by ReLU in a model whose weights are not yet retrained: the sparsity of the retrained activations will not change its latency. Unstructured pruning becomes a rewrite whose speedup can be measured in advance, in FP16 as in INT8. Every variant is timed on the same fixed input (uniform noise in the input range), so that comparisons are paired; only YOLO26's in-graph selection runs at a different speed on real images.

TABLE II
Latency change of a variant relative to the trained model on a fixed noise input, in % with 95 % CIs (NPU, two tiles; CNNs at 640 px, the mixed-INT8 ResNet-50 at 224 px). "Head": YOLO26-N without its final selection.

Model	Real images	Zero input	Random weights	50 % zero weights	90 % zero weights
ResNet-50	-0.2 ± 0.2	$+0.3 \pm 0.4$	$+0.1 \pm 0.4$	$\mathbf{-25.9} \pm 0.4$	$\mathbf{-35.1} \pm 0.3$
ResNet-50 mixed	$+1.4 \pm 0.7$	$+0.9 \pm 3.1$	-0.4 ± 0.5	$\mathbf{-23.5} \pm 0.7$	$\mathbf{-39.1} \pm 0.5$
MobileNetV2	$+1.0 \pm 1.3$	$+1.0 \pm 1.6$	$+1.0 \pm 0.8$	-1.9 ± 0.9	-4.6 ± 1.3
YOLO26-N FP16	$\mathbf{-5.6} \pm 0.5$	0.0 ± 0.5	$\mathbf{-4.6} \pm 0.2$	-6.0 ± 0.3	-8.6 ± 0.2
YOLO26-N INT8	$\mathbf{-5.6} \pm 0.3$	$+0.1 \pm 0.3$	$\mathbf{-3.3} \pm 0.3$	-6.1 ± 0.2	-6.6 ± 0.5
YOLO26-N head	-0.4 ± 0.7	-0.9 ± 0.9	-0.9 ± 0.8	-1.6 ± 0.7	-6.0 ± 0.7
D-FINE-N	$+0.5 \pm 0.3$	$+0.7 \pm 0.3$	$+0.1 \pm 0.3$	-2.2 ± 0.1	-2.5 ± 0.2

B. Measured profiler

Regions. The profiler attributes latency to *regions*, sets of runtime operations that come from the same source module (PyTorch module paths, or ONNX node scopes), truncated at a depth chosen to give a target number of regions. It also attributes latency to operation types.

Bypassing a region. To bypass a region, OptiNPU removes its operations and reconnects every tensor the region hands to the rest of the graph to the one tensor it receives with the same shape and type. The rewrite is refused when this mapping is ambiguous, or when it would create a cycle, which happens for a region that is not contiguous. Outputs that are no longer real are converted so that the graph stays valid. The saving of the bypassed graph is the latency of that region in context, fusions with its neighbors included. This saving is marginal: the compiler re-optimizes every bypassed graph, so the savings of the regions do not add up to the latency (O4). The profiler uses them to rank interventions, not to split the latency.

Paired, randomized, adaptive rounds. All variants and the reference are compiled in a single child process. Each round times every variant (30 to 40 inferences each) in a freshly shuffled order. A variant's saving is the mean of its per-round differences with the reference, with a 95 % confidence interval (normal approximation, ± 1.96 standard errors). Pairing cancels the drift of the machine. Rounds continue until every half-width is at most $\max(0.02\,\text{ms}, 0.5\,\%)$ of the latency, up to 40 rounds. A saving is *significant* when its lower bound is positive.

C. Task-level accuracy guard

The reference is always the original model on the CPU in FP32. The guard has two levels.

Agreement. Both deployments run on 30 guard images. The agreement is the F1 score of matching their detections: same class, IoU ≥ 0.7, confidence ≥ 0.3. Without annotations, a state is exact when its agreement is at least $\tau = 0.95$. For a classifier, the agreement is the share of images with the same top-1 class; for any other model, it is $1-$ the relative L2 error of the worst output. Table III calibrates the threshold on the two-tile faults of O5: any τ between 0.87 and 0.97

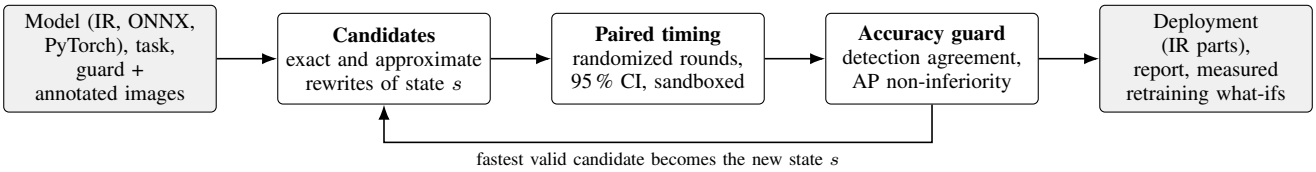


Fig. 1. The OptiNPU optimization loop. Every candidate is timed against the current state in the same sandboxed process, and checked against the CPU FP32 original before it is accepted. The loop stops when no candidate is accepted.

TABLE III
AGREEMENT WITH CPU FP32 ON 30 IMAGES, AGAINST THE REAL AP LOSS OF THE TWO-TILE MODE (FULL VAL2017).

	AP loss	Agreement	
Model	(2 tiles)	2 tiles	1 tile
RF-DETR-N	0.0	0.990	0.990
D-FINE-N	−0.2	0.919	0.994
DEIMv2-N	−0.9	0.862	0.975
RT-DETRv2-R18	−3.8	0.729	0.990
YOLO26-N	−4.6	0.842	0.989

TABLE IV
WHAT THE DESIGN CHOICES PREVENT: THE DECISION TAKEN WITHOUT EACH OF THEM, ON OUR MODELS.

Without	Outcome
agreement (max. output error instead)	RF-DETR-N (exact) is flagged, YOLO26-N (−4.6 AP) passes
the test (point estimate, 200 images)	a YOLO26-N mix within 0.3 points passes, then loses 0.9 on the report images; the test rejects it (−0.37, bound −1.2)
the confirmation	ResNet-50's mix (−0.00, bound −0.49) is kept and loses 0.64 on the report images; held-out images reject it (−0.90, bound −1.88)
20 folds (10 folds of 50 images)	15 ms paid to repair D-FINE-N's −0.25 (bound −0.64); with 20, its original is exact (−0.18, bound −0.44)
the *rest* group	D-FINE-N still loses 3.1 AP with 18 of 20 groups in float; *rest* alone recovers 2.85
approximate repairs, mixed precision on an inexact state	RT-DETRv2 takes the one-tile decoder (71.0 ms) instead of INT8 (40.6), which the confirmation later rejects

flags every fault that costs more than 0.5 AP, and we take 0.95. The calibration uses the evaluated models themselves, but τ only decides without annotations: none of the results of Section V-C depends on it.

AP non-inferiority. Agreement is only a proxy, and it errs both ways. The two-tile YOLO26-N reached 0.966 on one set of 30 images while losing more than 4 AP; D-FINE-N reaches only 0.923 while losing 0.2 AP. When annotations are given, the AP therefore decides: OptiNPU evaluates it on $K = 20$ interleaved folds of 50 images, for the candidate and for the CPU FP32 reference. With the per-fold differences d_k, it accepts the candidate when a one-sided paired t test rejects a loss larger than the budget B [37]:

$$\bar{d} - t_{0.95,\,K-1}\,\frac{s_d}{\sqrt{K}} \;\geq\; -B. \tag{1}$$

We use $B = 0.5$ AP. For a classifier, the same test applies to top-1 accuracy on a labeled set, with the same budget in accuracy points. This test decides whether the original deployment, a repair or an approximation counts as exact; with annotations, agreement only pre-filters the candidates.

Test power. Table IV shows what the test and its number of folds prevent. With 20 folds, the margins of approximate candidates close to the budget are 0.16 to 0.45 points on the seven detectors, and D-FINE-N's original deployment (-0.18, lower bound -0.44) is now exact. Top-1 on 2 000 ImageNetV2 images is noisier, with margins of 0.37 to 0.92 points: a classifier mix passes only when it shows almost no loss.

Confirmation. The optimizer tests several candidates on the same decision images, and the mixed-precision search tests growing mixes until one passes. Keeping the first success among noisy tests favors an underestimated loss. An approximation that passes Eq. (1) must therefore pass it again on held-out images, disjoint from all others, before it is accepted. A mix that fails ends the search: trying the next one would select on the held-out images too. Exact rewrites and repairs are not confirmed, since they are not meant to change the function. The confirmation removes the selection bias at a cost in power, since a candidate must pass two tests. On 1 000 held-out COCO images, the margins are 0.19 to 0.28 points, and a candidate that truly loses 0.25 points fails the confirmation in 30 to 60 % of the cases. On 2 000 ImageNetV2 images, the margins (0.67 to 0.98 points) exceed the budget: a mix without any loss passes the confirmation in only 20 to 30 % of the cases.

D. Optimizer

State. The optimizer searches over deployment states $s = (\text{graph}, \text{compile options}, \text{placement})$. The placement either runs the whole graph on the NPU, or cuts it into two IR parts, each with its own device and options.

Target. The network stays on the NPU: the point of an NPU is to leave the CPU and the iGPU free, at low power. Only a *light selection tail* may move to the CPU. This tail is the first `TopK` or NMS and everything that depends on it, provided it has no convolution or matrix product; this is the host post-processing every deployment performs. Moving heavier parts, such as a DETR decoder, to the iGPU or the CPU is an explicit option that we use only as a reference point (Section V-G).

Candidates. Table V lists the rewrites, in three tiers:

- *exact* rewrites do not change the function;
- *approximate* rewrites change it on purpose and must keep AP within the budget;

TABLE V
REWRITE CATALOG.

Tier	Rewrites
Exact	final selection on the host CPU; one or two tiles; DETR decoder compiled apart on one tile; depthwise as dense block-diagonal convolution
Approximate	INT8 post-training quantization (NNCF CNN and transformer schemes); latency-guided mixed precision
Retraining	SiLU/HardSwish/GELU $\rightarrow$ ReLU; LayerNorm $\rightarrow$ BatchNorm; 50 % and 75 % unstructured pruning

- *retraining* rewrites are only timed on the final state, and reported with their measured latency.

Greedy loop (Algorithm 1). At each step, all candidates are timed against the current state in one paired session. They are then checked in decreasing order of saving. The first candidate that is significantly faster (lower bound above zero and saving above 1 %) and passes the guard is applied. Approximate candidates are always judged against the CPU FP32 original, never against the current state, so their errors do not accumulate. The loop stops when no candidate is accepted, and after five steps at most; no run needed more than two.

Repairs. An inexact state (O5) accepts any candidate that repairs it, even a slower one: correctness comes first. A rewrite of the exact tier that is faster and agrees better, without passing the guard, is also accepted. Approximate candidates can repair too; for instance, INT8 removes the two-tile fault of RT-DETRv2. On an inexact state, mixed precision is searched only when full INT8 keeps the mean loss within the budget and misses only its lower bound, since keeping groups in float could bring the fault back.

Latency-guided mixed precision. The search starts when full INT8 is significantly faster but outside the AP budget. It forms *groups* of operations to keep in float:

- one group per module region that contains a convolution or a matrix product, with about ten groups in total;
- one group, *rest*, for every other quantized operation. Without it, quantizers on a DETR decoder's box arithmetic can never return to float (Section V-I2).

Each group is kept alone in float on top of full INT8, quantized with 64 calibration images. Its AP gain is measured on the annotated set, and its latency cost in a paired session against full INT8. Among the groups that cost more than a quarter of the INT8 saving, the most useful one that can be split is split into its sub-modules and measured again, up to three times. Groups are then ranked by AP gained per millisecond. Growing mixes are quantized with all 300 calibration images and *verified as a whole*, for latency and with Eq. (1). The search stops at the first mix that passes, which must then be confirmed, or when adding float groups removes the significant gain. We never assume that the latency of a mix is the sum of its groups' costs.

Algorithm 1 Measured greedy optimization

Require: model M, task, guard images, annotated set, AP budget B

$s \leftarrow (M,$ default options, whole graph on the NPU$)$
$x \leftarrow \textsc{Exact}(s)$ ▷ Eq. (1), else agreement $\geq \tau$
for at most five steps **do**
  $C \leftarrow \textsc{Candidates}(s)$
  time s and C in paired rounds: savings $\Delta_c \pm h_c$
  $s' \leftarrow \bot$
  for $c \in C$ in decreasing order of Δ_c **do**
    $f \leftarrow \Delta_c - h_c > 0$ **and** $\Delta_c > 1\,\%$ ▷ faster
    if x **and not** f **then**
      continue ▷ an exact state only accepts gains
    end if
    $ok \leftarrow \textsc{Exact}(c)$
    if c approximate **and not** ok **and** (x **or** mean loss $\leq B$) **then**
      $(ok, c) \leftarrow \textsc{MixedPrecision}(s, c)$ ▷ first mix that passes
    end if
    if c approximate **and** ok **then**
      $ok \leftarrow \textsc{Confirmed}(c)$ ▷ held-out images
    end if
    if ok **or** (c exact-tier **and not** x **and** f **and** c agrees better than s) **then**
      $s' \leftarrow c$; **break**
    end if
  end for
  if $s' = \bot$ **then**
    break
  end if
  $s \leftarrow s'$; $x \leftarrow \textsc{Exact}(s)$
end for
return s and the measured latency of the retraining rewrites on s

E. Implementation

OptiNPU is a Python library and command-line tool (`optinpu profile`, `optinpu optimize`) built on OpenVINO and NNCF. Every compilation and inference runs in a child process watched for memory and time, whose whole process tree is killed on failure. On this NPU, the compiler reached 53 to 55 GB of virtual memory on some graphs, and a lost device must become a rejected candidate rather than a crash of the tool. The optimized deployment is saved as IR parts with a JSON description (devices, compile options, applied changes) and a report. It can also be rebuilt from its list of changes: the seven rebuilt detector deployments reproduce their recorded AP exactly.

V. EVALUATION

A. Setup

Hardware and software. We use an Intel Core Ultra 5 235U laptop (NPU 3720, driver 32.0.100.4778, 15.5 GB of

RAM) on mains power, with Windows 11 (Balanced power plan), OpenVINO 2026.4.1 and NNCF 3.4. A latency is the time of one synchronous batch-1 inference of the whole deployment, timed as in Section IV-B: all parts back to back, transfers and the CPU tail included, image decoding and resizing excluded.

Models. We use seven detectors, chosen to cover the architectural families that behave differently on the NPU:

- YOLO26-N and -S [8]: CNNs with an in-graph `TopK`, no NMS;
- D-FINE-N and -S [4], and DEIMv2-N [6]: HGNetv2 backbones and deformable decoders;
- RT-DETRv2-R18 [3]: a ResNet backbone and a standard attention decoder;
- RF-DETR-N [7]: a DINOv2 ViT backbone [38].

They are exported as FP16 IRs. To show that the approach is not specific to detection, we also optimize three ImageNet classifiers [39] at 224 px, with their torchvision weights: ResNet-50 [40], MobileNetV2 [41] and EfficientNet-B0 [42]. The profiler validation uses the same three CNNs at 640 px.

Protocol. All detectors were trained on COCO train2017, so every decision is taken on val2017 images disjoint from the reported ones: guard [4970, 5000), INT8 calibration [4500, 4800), AP budget [2500, 3500) in 20 folds of 50 images, and confirmation [3500, 4500), also in 20 folds. We report AP on val2017 [0, 2500), on which nothing is decided. The classifiers are judged on ImageNetV2 [43] (matched frequency, 10 000 images), which none of them was trained on. We split one fixed shuffle of it into disjoint slices: guard [7300, 7600) (top-1 agreement), calibration [7000, 7300), top-1 budget [5000, 7000) in 10 folds of 200, confirmation [7600, 9600), also in 10 folds, and report [0, 5000). Unless noted, the latencies of a model's deployments come from one final paired session, timed against the original on the NPU in up to 60 rounds, with 95 % confidence intervals.

B. Does measured profiling pick better interventions?

Ground truth. On five models we apply 150 interventions: bypassing a region or an operation type, SiLU $\rightarrow$ ReLU, removing LayerNorms, and moving the final selection to the CPU. We measure each intervention's saving twice, in opposite orders, with up to 120 adaptive rounds. Run 1 is the ground truth; run 2 is an independent repetition.

Predictors. We compare four ways to predict these savings:

- *vendor counters*: the durations of the removed operations, scaled to the latency;
- an *additive* model in the style of nn-Meter: each operation compiled alone on the NPU, minus the cost of a graph reduced to its inputs;
- a *learned* predictor: a ridge regression trained, for each model, on the run-1 savings of the four other models, from the two predictions above, the number of removed operations and the kind of intervention;
- *our measurement*: run 2, which shows how reproducible one measured run is and bounds what any predictor can reach.

TABLE VI
PREDICTING THE SAVING OF AN INTERVENTION, AGAINST RUN 1: MEAN OVER FIVE MODELS (92 SIGNIFICANT INTERVENTIONS OUT OF 150).

Predictor	ρ	Error (% lat.)	Top-3	Regret
Vendor counters (scaled)	$+0.36$	3.2	0.60	0.31
Additive (isolated ops)	$+0.49$	9.5	0.67	0.60
Learned (other models)	$+0.28$	3.8	0.53	0.21
Measured (run 2)	$\mathbf{+0.84}$	**0.8**	**0.87**	**0.00**

Metrics. Over the 92 interventions that are significant in run 1, we report Spearman's ρ, the mean absolute error as a percentage of the latency, the overlap of the top three interventions, and the *regret*: the share of the best saving lost by applying the predictor's first choice.

Table VI gives the results. Run 2 never misses the best intervention of run 1: its regret is 0 on every model. The vendor counters have a regret of 0.63 on D-FINE-N and 0.78 on EfficientNet-B0. The additive model has a regret of 0.99 on YOLO26-N and 0.78 on ResNet-50; on YOLO26-N, both baselines even rank the interventions in the wrong order ($\rho = -0.04$ and -0.10). Fitting a predictor to measurements of this NPU does not help: the learned one ranks worse than both (regret 0.94 on D-FINE-N), and a gradient-boosted regressor does no better (regret 0.31). With five models its training data is small, but it would have to learn decisions the compiler takes on the whole graph (O4). This accuracy has a cost: one measured run of all the interventions of a model takes 1.8 to 12 minutes (36 minutes for the five models), against 2 to 4 s for the vendor counters and 0.6 to 2.3 minutes for the additive model. Summed over all operations of the four larger models, the counters give 123 to 297 ms and the additive model 72 to 260 ms, for real latencies of 17 to 31 ms.

Limit. On the smallest model (MobileNetV2, 7.0 ms), 21 of the 23 savings are significant but close to each other, and their fine order stays at the noise level even after 120 rounds: the repeated measurement ranks them less consistently ($\rho = +0.68$) than the vendor counters ($+0.74$), but still picks the best one (regret 0, against 0.12).

C. Optimizing detectors for the NPU

Table VII reports the optimizer with the network on the NPU (Section IV-D).

YOLO26. The optimizer moves 16 operations, the final selection, to the CPU. This makes YOLO26-N 2.84$\times$ faster and YOLO26-S 2.02$\times$ faster on the NPU (YOLO26-N about 2.7$\times$ on real images, on which its in-graph selection runs 5.6 % faster, Table II). Exporting a detector without its final selection is a common deployment practice. The measurement adds that this tail costs two thirds of the latency (Section V-I), and the guard shows that removing it also removes both two-tile faults: the silent loss of 4.6 AP of YOLO26-N, and the device hang of YOLO26-S. The tail compiled alone on two tiles does not hang, so the faults arise when the compiler combines the selection with the rest of the graph. Mixed INT8 would then save up to 2.7 ms more on YOLO26-S, but every

TABLE VII

OPTIMIZER RESULTS, NETWORK ON THE NPU, WITH THE CONFIRMATION (FP16 IRS; AP ON VAL2017 [0, 2500), ON WHICH NOTHING WAS DECIDED). LATENCIES FROM ONE PAIRED SESSION PER MODEL: "BEFORE" IS THE DEFAULT TWO-TILE COMPILATION; "AFTER" IS BEFORE MINUS THE PAIRED SAVING, WITH ITS 95 % HALF-WIDTH.

Model	Before	After	Applied changes	AP: CPU FP32 / before / after
YOLO26-N	26.7 ms	**9.40 ± 0.09 ms (2.84×)**	final selection on the CPU	41.7 / 37.0 / 41.6
YOLO26-S	33.0 ms, device lost	**16.35 ± 0.16 ms (2.02×)**	final selection on the CPU	49.4 / — / 49.4
D-FINE-N	26.0 ms	26.0 ms (1.00×)	none: the original is exact	44.5 / 44.3 / 44.3
D-FINE-S	46.9 ms	46.9 ms (1.00×)	none: mixed INT8 not confirmed*	50.2 / 50.1 / 50.1
DEIMv2-N	23.9 ms	41.01 ± 0.03 ms (0.58×)	repair: one tile (original AP −0.44, bound −0.69)	45.0 / 44.0 / 44.9
RT-DETRv2-R18	47.4 ms	76.09 ± 0.08 ms (0.62×)	repair: decoder on one tile; INT8 not confirmed*	49.7 / 46.0 / 49.7
RF-DETR-N	83.3 ms	81.82 ± 0.06 ms (1.02×)	INT8, transformer scheme	50.0 / 50.0 / 49.7

*Without the confirmation: D-FINE-S 42.9 ms (1.08×, AP 49.9) with 7 of 19 groups in float; RT-DETRv2 40.6 ms (1.17×, AP 49.6) with INT8 in the transformer scheme.

mix loses about 0.5 AP (lower bounds −0.73 to −0.89) and is rejected, as on YOLO26-N.

DETR family. Without leaving the NPU, the DETRs mostly need repairs, and the guard decides what they may cost:

- *RT-DETRv2* loses 3.7 AP in two-tile mode on the report images. INT8 with the transformer scheme repairs it and is faster (40.6 ms), within 0.1 AP of the CPU on the report images, but it loses 0.47 AP on the held-out images (bound −0.72) and is not confirmed. The optimizer then takes an exact repair, the decoder compiled apart on one tile: 76.1 ms, with the AP of the CPU (Section V-I3).
- *DEIMv2-N* loses 0.44 AP on the decision images (lower bound −0.69), and 1.0 on the report images. Neither INT8 scheme stays within the budget (−22.5 and −1.2 AP), and the decoder alone on one tile is slower than the whole graph on one tile. The cheapest exact deployment therefore runs on one tile: 41.2 ms instead of 24.0. With a budget of one point, the original would be kept; the report gives both.
- *D-FINE-N* loses only 0.18 AP: the original is exact, and nothing else is both faster and within the budget. Full INT8 saves only 0.3 ms and costs 5.9 AP, most of it in the backbone.
- *D-FINE-S* and *RF-DETR-N* are exact on the NPU. Full INT8 costs D-FINE-S 6.0 AP; keeping 7 of 19 groups in float, the *rest* group included, brings it within the budget (−0.13, bound −0.36), and this mix is even 1.3 ms faster than full INT8. The held-out images do not confirm it (−0.25, bound −0.52), and the original is kept; on the report images, the mix loses 0.27 AP, within the budget but not certified. RF-DETR-N gains only 2 %, with INT8 in the transformer scheme, confirmed (−0.22, bound −0.41); it loses 0.27 AP on the report images.

On the NPU alone, none of them beats its best placement of Table I.

D. Beyond detection: image classifiers

The same optimizer runs on classifiers, with the top-1 agreement as guard and top-1 accuracy as the judged metric (Table VIII). On the report slice, CPU FP32 top-1 is 11 to 13 points below each model's ImageNet accuracy, as expected [43]. The compiler is exact on all three (within 0.15 points of the CPU), so only INT8 can make them faster. On ResNet-50, full INT8 is 1.62× faster but loses 0.30 points (bound −0.90); keeping 5 of 20 groups in float passes the test (−0.00, bound −0.49) and keeps 1.43×. On MobileNetV2, full INT8 saves 19 % and loses 0.35 points; four groups in float pass (+0.05, bound −0.47) at almost no cost. On EfficientNet-B0, full INT8 saves only 6 % and loses 1.25 points, no group recovers more than 0.2 points when kept in float, and the original is kept. Depthwise convolutions rewritten as dense ones are slower on both mobile networks.

TABLE VIII

CLASSIFIERS, NETWORK ON THE NPU (224 PX), IN THE OPTIMIZER'S RUN WITH THE CONFIRMATION: THE MIX THAT PASSES EQ. (1), AND ITS TOP-1 CHANGE AGAINST CPU FP32 IN POINTS (95 % LOWER BOUND) ON THE DECISION AND HELD-OUT IMAGES. THE ORIGINAL IS KEPT ON ALL THREE; ON IMAGENETV2 [0, 5000), IT IS WITHIN 0.15 POINTS OF CPU FP32.

	Before	Mix	Top-1 change (bound)	
Model	(ms)	(ms)	decision	held-out
ResNet-50	4.52	3.16 (1.43×)	−0.00 (−0.49)	−0.90 (−1.88)
MobileNetV2	1.40	1.14 (1.23×)	+0.05 (−0.47)	+0.15 (−0.53)
EfficientNet-B0	2.75	none	—	—

Neither mix is confirmed, and the original is kept on all three classifiers. ResNet-50's mix loses 0.90 points on the held-out images (bound −1.88) and 0.64 on the report images: the search had tested five growing mixes on the same images and kept the first that passed, after four with point estimates of −0.05 to −0.30, which favors an underestimated loss. MobileNetV2's mix gains 0.15 points on the held-out images, but its bound (−0.53) misses the budget: 2 000 images cannot certify a loss below half a point. An earlier run without the confirmation had kept a mix on both: 1.60× and 1.22×, with −0.64 and +0.1 points on the report images.

E. Against NNCF's accuracy-aware quantization

NNCF's `quantize_with_accuracy_control` [23] is the closest existing tool: starting from full INT8, it returns to float, one at a time, the quantizers that hurt the metric most, until the drop on a validation set is at most a given

TABLE IX
NNCF'S ACCURACY-AWARE QUANTIZATION AGAINST OPTINPU, ON THE NPU: SEARCH TIME, LATENCY (ONE PAIRED SESSION) AND METRIC CHANGE AGAINST CPU FP32 ON THE REPORT IMAGES, IN POINTS.

	NNCF			OptiNPU	
Model	Search	ms	Δ	ms	Δ
ResNet-50	2 min	1.99	−0.4	3.81*	0.0
MobileNetV2	1 min	0.79	−0.2	1.03*	0.0
EfficientNet-B0	25 min	2.60	**−1.5**	2.77	0.0
YOLO26-N	82 min	8.51	**−18.2**	9.20	−0.1
YOLO26-S	146 min	13.69	−0.3	16.27	0.0
D-FINE-S	>6 h	budget not reached		46.9*	−0.1

*The original: Eq. (1) alone accepts a mix (2.40 ms and −0.6; 0.81 and 0.0; 42.9 and −0.3), which the confirmation rejects.

value. We give it what OptiNPU's search had: the same graph (for YOLO26, the network without its selection), calibration images, decision images and budget (0.5 point); it validates on the CPU. Table IX compares the results on the NPU.

None of NNCF's models passes Eq. (1) on the decision images: a point estimate accepts losses that 1 000 images cannot certify. Where its model stays within the budget on the report images (ResNet-50, MobileNetV2, YOLO26-S), it is faster than OptiNPU's deployment, which keeps the original classifiers once the confirmation has rejected their mixes: 1.99 against 3.81 ms on ResNet-50, 0.79 against 1.03 on MobileNetV2, and 13.69 against 16.27 on YOLO26-S. Our test is the more conservative: it refuses losses it cannot certify, and here this costs three models their INT8 gain. Validating on the CPU, however, misses the NPU: EfficientNet-B0 loses 1.5 points and YOLO26-N 18.2 AP on the NPU, although both stayed within the budget on the CPU. The INT8 network of YOLO26-N without its selection is wrong on the NPU only, a fault that no check on the CPU can see. On D-FINE-S, NNCF had not reached the budget after 6 h: after 38 iterations, it still lost 3.4 AP, spread over the decoder's box arithmetic (Section V-I2).

F. Reference point: the iGPU, optimized the same way

Table I compares the NPU with the original model on the iGPU. For a fair comparison, we also run OptiNPU for the iGPU, with the same rewrites, budget, images and confirmation, and time both optimized deployments in the final session (Table X). INT8 works far better on the iGPU: it is confirmed on four of the seven detectors (1.28 to 1.78× faster than the original on the iGPU), against one on the NPU (1.02×). D-FINE-N and -S run 1.36 and 1.74× faster with INT8 in the transformer scheme and their backbone in FP16, while on the NPU this scheme makes D-FINE-S slower (53.0 against 47.1 ms). RT-DETRv2's INT8 is rejected by the confirmation on both devices (−0.47 and −0.58 AP on the held-out images). Optimized this way, the iGPU is faster than the NPU on seven of the ten models, by 1.3 to 4.5×; the NPU stays faster on YOLO26-S (16.4 against 20.2 ms) and ResNet-50 (3.75 against 4.33 ms). MobileNetV2 is a tie: its latency on the NPU varied from 1.1 to 1.6 ms between sessions, more than its difference with the iGPU.

TABLE X
NPU AGAINST IGPU, BOTH OPTIMIZED BY OPTINPU WITH THE CONFIRMATION AND TIMED IN ONE PAIRED SESSION PER MODEL (95 % HALF-WIDTHS 0.03 TO 0.18 MS): LATENCY IN MS, AND IN PARENTHESES THE METRIC CHANGE AGAINST CPU FP32 ON THE REPORT IMAGES, IN POINTS. —: NOTHING APPLIED.

	NPU	iGPU		
Model	optimized	original	optimized	changes
YOLO26-N	9.40 (−0.1)	11.19	**6.29** (−0.5)	mixed INT8
YOLO26-S	**16.35** (0.0)	23.53	20.21 (0.0)	selection on CPU
D-FINE-N	25.97 (−0.2)	17.55	**13.86** (−0.1)	INT8*
D-FINE-S	46.93 (−0.1)	43.02	**25.16** (−0.1)	INT8*
DEIMv2-N	41.01† (−0.1)	**17.14**	—	
RT-DETRv2-R18	76.09† (0.0)	**47.58**	—	
RF-DETR-N	81.82 (−0.3)	23.39	**18.16** (−0.3)	INT8‡
ResNet-50	**3.75** (0.0)	4.33	—	
MobileNetV2	1.59 (0.0)	**1.29**	—	
EfficientNet-B0	2.81 (0.0)	**2.10**	—	

*transformer scheme, backbone in FP16. ‡transformer scheme. †repaired.

TABLE XI
WITH THE DECODER ALLOWED ON THE IGPU: OPTINPU'S NPU+IGPU DEPLOYMENT IN ITS OWN RUN AND IN THE FINAL SESSION (± 95 % HALF-WIDTH), ITS AP ON VAL2017 [0, 2500), AND THE IGPU ALONE IN THE FINAL SESSION (ORIGINAL / OPTIMIZED), IN MS.

	NPU+iGPU		AP CPU	iGPU
Model	own run	final	/ after	orig. / opt.
D-FINE-N	15.2	17.5 ± 0.5	44.5 / 44.4	17.6 / 13.9
D-FINE-S	30.3‡	35.0 ± 0.5	50.2 / 50.0	43.0 / 25.2
DEIMv2-N	16.6	16.0 ± 0.6	45.0 / 44.9	17.1 / —
RT-DETRv2-R18	26.1*†	32.3 ± 0.3	49.7 / 49.6	47.6 / —
RF-DETR-N	40.1†	39.3 ± 0.4	50.0 / 49.7	23.4 / 18.2

*cut at the end of the encoder. †+ INT8 (transformer scheme). ‡+ mixed INT8; approximations accepted by Eq. (1) alone, without the confirmation.

G. Reference point: offloading the decoder

When the decoder may leave the NPU, the optimizer moves it to the iGPU on every DETR, 1.4 to 2.1× faster than the original on the NPU in its own run (Table XI); there, it matched or beat the fastest placement of Table I on D-FINE-N, D-FINE-S and RT-DETRv2. Hybrid latencies are less stable than the others: timed again in the final session, the same deployments ran from 4 % faster to 24 % slower, while the originals on the NPU stayed within 1.5 % of their earlier latency. In that session, the hybrid deployment beats the original on the iGPU on D-FINE-S, DEIMv2-N and RT-DETRv2, ties on D-FINE-N and loses on RF-DETR-N; the optimized iGPU deployment is faster on D-FINE-N, D-FINE-S and RF-DETR-N. Three hybrid deployments also rely on approximations accepted without the confirmation, which rejected RT-DETRv2's INT8 on both devices. This is why OptiNPU keeps the network on the NPU by default: offloading competes with other devices rather than optimizing for the NPU.

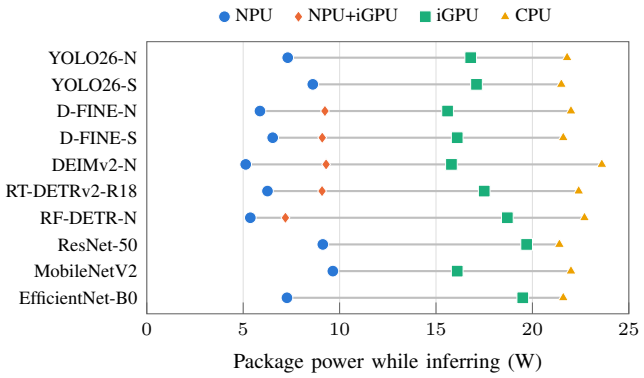


Fig. 2. Package power while inferring back to back: OptiNPU's NPU deployment (Tables VII and VIII), its NPU+iGPU deployment (Table XI), the iGPU deployment (optimized when something was applied, Table X), and the original on the CPU (FP32). At idle, the package draws 2.5 to 3.1 W.

H. Energy per inference

Windows exposes the RAPL energy counters of the whole package (CPU cores, iGPU, NPU and the rest of the SoC); the NPU has no counter of its own. We measure the package power during three 20 s windows of back-to-back inferences, each after 4 s at idle, with the display allowed to turn off (Fig. 2): 5.1 to 9.7 W on the NPU, 7.2 to 9.3 W with the decoder on the iGPU, 15.6 to 19.7 W on the iGPU and 21.4 to 23.6 W on the CPU, against 2.5 to 3.1 W at idle. Multiplied by the latency, it gives the energy of one inference. On nine of the ten models, the NPU deployment uses 1.2 to 2.3× less energy than the iGPU deployment, even where the iGPU is faster, and the CPU 3.9 to 17× more. The exception is RF-DETR-N, whose optimized iGPU deployment is 4.5× faster and uses 1.3× less energy. On RF-DETR-N, and on RT-DETRv2 and DEIMv2-N, whose NPU deployments are slow repairs, the NPU+iGPU deployment uses the least energy. Counting only the power above idle, which is noisier, the NPU uses 1.3 to 3.5× less energy than the iGPU on all ten models. `NPU_TURBO` changes the package power by -0.2 to $+0.4$ W and the energy per inference by -4 to $+5\,\%$: its speedup costs no measurable energy. A first campaign, with 8 s windows and the display on (7 to 10 W at idle), gave higher energies and the same ranking, except on DEIMv2-N, whose one-tile deployment then used more energy than the iGPU.

I. Case studies

1) Where the time goes in YOLO26-N: Bypassing the final selection (16 operations) saves 17.5 ms, two thirds of the 26.4 ms latency (Section V-B). The vendor counters point at the same operations, which run on the SHAVE cores (`Divide` 19.5 ms, `GatherElements` 15.4 ms, `TopK` 4.1 ms), but their durations overlap: over the 16 operations, they add up to 58.7 ms, twice the whole inference. Only the bypass measurement gives the real saving, which the selection-to-CPU rewrite then achieves (17.3 ms).

2) Mixed precision is not additive: **The loss is concentrated.** On YOLO26-S, full INT8 loses 6.6 AP. Most of the loss sits in one attention block: keeping it in float recovers 4.5 AP for 0.04 ms.

Latency is not additive. On RF-DETR-N, whose backbone runs on the NPU, NNCF's CNN scheme loses 4.2 AP, and the optimizer refined the search down to the 12 DINOv2 layers. Keeping the *whole* encoder stack in float costs 4.2 ms. Keeping a *single* middle layer in float, between INT8 layers, costs 13 to 16 ms. Precision changes and the re-tiling between layers of different precisions cost more than the arithmetic. A cost model that sums per-layer costs would choose the single layer; measuring the whole mix avoids that mistake.

Module groups miss operations. On D-FINE-N, keeping 18 of 20 module groups in float still left a 3.1-point loss (Table IV). With every group in float, 64 quantizers remained, almost all on the decoder's box and reference-point arithmetic, which belongs to no module containing a convolution. The *rest* group fixes this, and it becomes the most useful group of D-FINE-N: keeping it in float recovers 2.85 AP and even saves 0.47 ms.

3) Repairing on the NPU: On RT-DETRv2, the two-tile fault sits in the decoder's self-attention. Rewriting the attention in the IR changes nothing, because the compiler normalizes all its variants. Compiling the decoder apart on one tile, with the rest on two, repairs the model on the NPU (agreement 0.721 $\rightarrow$ 0.995), but costs 76.1 ms instead of 47.4: cutting the graph into two NPU inferences alone costs 8.9 ms, and one tile slows the decoder down by 19 ms. INT8 with the transformer scheme also removes the fault, because the compiler then partitions the attention differently, and it is faster than the original (40.6 ms). Two rules let the optimizer consider this repair (Section IV-D): approximate candidates may repair an inexact state, and mixed precision may be searched on an inexact state when full INT8 keeps the mean loss within the budget. The INT8 repair passes Eq. (1) on the decision images (-0.11, bound -0.37) and loses only 0.1 AP on the report images, but 0.47 on the held-out images (bound -0.72), and 0.58 on the iGPU: the confirmation rejects it, and the optimizer pays 35 ms for the repair it can certify.

J. Retraining rewrites, measured before training

Thanks to the premise, OptiNPU reports the exact latency of rewrites that need retraining: SiLU $\rightarrow$ ReLU would make the optimized YOLO26-N 21 % faster (7.33 ms), YOLO26-S 17 % faster (13.4 ms) and EfficientNet-B0 35 % faster (1.79 ms); pruning 50 % or 75 % of the weights would make ResNet-50 18 % or 33 % faster (3.71 or 3.03 ms, from 4.52); on the five DETR-family models, the best of these rewrites brings only 3 to 9 %. OptiNPU does not retrain the model: it tells, before any training, which retraining is worth its cost. A gain of 17 to 35 % justifies fine-tuning YOLO26 or EfficientNet-B0 with ReLU; 3 to 9 % hardly justifies retraining a DETR.

VI. Discussion and Limitations

One NPU. We measured a single NPU (3720), with a single driver and compiler version, and did not test OptiNPU on another NPU. Its faults and costs may differ on other NPUs

or change with a driver update; this is precisely why OptiNPU measures instead of encoding rules. Porting it to another NPU supported by OpenVINO only requires a device name; another runtime would need new sandboxed jobs.

Accuracy guard. The guard relies on samples. Agreement on 30 images can miss AP losses (Section IV-C), so AP decides when annotations exist; without them, an approximate rewrite must reach the agreement of an exact one, a conservative rule. The test averages AP over folds of 50 images; an image-level bootstrap of the AP would not assume normal fold differences. The confirmation removes the bias of keeping the first of several passing mixes, but costs power: it rejected three candidates whose mean loss stays within the budget (at most 0.47 points on the held-out images, and 0.27 on the report images where we measured it). More decision and held-out images would let such candidates pass; on the classifiers, 2 000 held-out images are too few to confirm even a mix without any loss reliably. Every result uses $B = 0.5$ AP; a budget of one point would keep DEIMv2-N's original (24.0 ms) and accept YOLO26-S's mixes.

Cost of the search. The search applies one change at a time, so it can miss combinations that only pay off together. A whole run, final evaluation included, takes 12 to 24 minutes for a classifier and 35 minutes to 1 h 50 for a detector on our laptop. Re-running the optimizer reproduced its decisions up to the confirmation, with the same mixes on D-FINE-S and ResNet-50; only MobileNetV2, whose groups cost about 0.01 ms each, got a different mix. The optimizer explores one compile option (`NPU_TILES`) and batch-1 latency. Measured on its deployments, other options change little: `NPU_TURBO` saves up to 6.6 %, most on the smallest models, at no measurable energy cost (Section V-H), and the throughput hint and QDQ optimization nothing significant. With two inferences in flight, throughput rises by 2 to 9 % on the detectors, by 23 to 63 % on the classifiers, and doubles on the one-tile DEIMv2-N, whose two inferences then use a tile each; overlapping YOLO26's CPU tail with the next inference adds only 7 and 3 %. Beyond detectors and classifiers, other models can use the generic output-agreement task, without a metric test.

VII. Conclusion

On the client NPU we studied, latency could not be predicted from FLOPs, vendor counters, per-operation costs or a regressor fitted to them, and the compiler was silently wrong. Two facts make a measurement-only approach practical: latency does not depend on dense weight values, and paired, randomized, adaptive rounds measure small savings reliably. OptiNPU turns these facts into a profiler that picks the right intervention, an accuracy guard grounded in the task, and an optimizer that repairs and accelerates detectors without leaving the NPU. It repairs the four detectors that the compiler gets wrong and makes YOLO26-N and -S 2.84 and 2.02× faster (YOLO26-S stays ahead of the iGPU even when the iGPU is optimized the same way); checking accuracy on the CPU instead would have missed an 18-AP loss. Once approximations must hold on held-out images, INT8 helps little on this NPU: one DETR gains 1.02× and no classifier mix is confirmed, while the same optimizer confirms INT8 on four detectors on the iGPU. The NPU's advantage is then energy rather than speed: it uses less energy per inference than the iGPU on nine of the ten models. The measured what-ifs tell which retraining is worth its cost.

Acknowledgment

The author used AI tools to help gather the references and write this paper, and reviewed all of its content.

References

[1] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. ECCV*, 2020.

[2] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "DETRs beat YOLOs on real-time object detection," in *Proc. CVPR*, 2024.

[3] W. Lv, Y. Zhao, Q. Chang, K. Huang, G. Wang, and Y. Liu, "RT-DETRv2: Improved baseline with bag-of-freebies for real-time detection transformer," arXiv:2407.17140, 2024.

[4] Y. Peng, H. Li, P. Wu, Y. Zhang, X. Sun, and F. Wu, "D-FINE: Redefine regression task in DETRs as fine-grained distribution refinement," in *Proc. ICLR*, 2025.

[5] S. Huang, Z. Lu, X. Cun, Y. Yu, X. Zhou, and X. Shen, "DEIM: DETR with improved matching for fast convergence," in *Proc. CVPR*, 2025.

[6] S. Huang, Y. Hou, L. Liu, X. Yu, and X. Shen, "Real-time object detection meets DINOv3," arXiv:2509.20787, 2025.

[7] I. Robinson, P. Robicheaux, M. Popov, D. Ramanan, and N. Peri, "RF-DETR: Neural architecture search for real-time detection transformers," in *Proc. ICLR*, 2026.

[8] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, and M. E. Kalfaoglu, "Ultralytics YOLO26: Unified real-time end-to-end vision models," arXiv:2606.03748, 2026.

[9] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *Proc. ECCV*, 2014.

[10] L. L. Zhang, S. Han, J. Wei, N. Zheng, T. Cao, Y. Yang, and Y. Liu, "nn-Meter: Towards accurate latency prediction of deep-learning model inference on diverse edge devices," in *Proc. ACM MobiSys*, 2021.

[11] C. Li *et al.*, "HW-NAS-Bench: Hardware-aware neural architecture search benchmark," in *Proc. ICLR*, 2021.

[12] Ł. Dudziak, T. Chau, M. S. Abdelfattah, R. Lee, H. Kim, and N. D. Lane, "BRP-NAS: Prediction-based NAS using GCNs," in *Proc. NeurIPS*, 2020.

[13] T.-J. Yang, A. Howard, B. Chen, X. Zhang, A. Go, M. Sandler, V. Sze, and H. Adam, "NetAdapt: Platform-aware neural network adaptation for mobile applications," in *Proc. ECCV*, 2018.

[14] H. Cai, L. Zhu, and S. Han, "ProxylessNAS: Direct neural architecture search on target task and hardware," in *Proc. ICLR*, 2019.

[15] M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, and Q. V. Le, "MnasNet: Platform-aware neural architecture search for mobile," in *Proc. CVPR*, 2019.

[16] Y. He, J. Lin, Z. Liu, H. Wang, L.-J. Li, and S. Han, "AMC: AutoML for model compression and acceleration on mobile devices," in *Proc. ECCV*, 2018.

[17] Z. Yao *et al.*, "HAWQ-V3: Dyadic neural network quantization," in *Proc. ICML*, 2021.

[18] Intel Corporation, "OpenVINO toolkit," https://github.com/openvinotoolkit/openvino, 2026, version 2026.4.1.

[19] Y. Xiong, H. Liu, S. Gupta, B. Akin, G. Bender, Y. Wang, P.-J. Kindermans, M. Tan, V. Singh, and B. Chen, "MobileDets: Searching for object detection architectures for mobile accelerators," in *Proc. CVPR*, 2021.

[20] S. Gupta and B. Akin, "Accelerator-aware neural network design using AutoML," arXiv:2003.02838, 2020.

[21] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han, "HAQ: Hardware-aware automated quantization with mixed precision," in *Proc. CVPR*, 2019.

[22] Z. Dong, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer, "HAWQ: Hessian AWare quantization of neural networks with mixed-precision," in *Proc. ICCV*, 2019.

[23] A. Kozlov, I. Lazarevich, V. Shamporov, N. Lyalyushkin, and Y. Gorbachev, "Neural network compression framework for fast model inference," arXiv:2002.08679, 2020.

[24] T. Chen *et al.*, "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proc. USENIX OSDI*, 2018.

[25] L. Zheng *et al.*, "Ansor: Generating high-performance tensor programs for deep learning," in *Proc. USENIX OSDI*, 2020.

[26] Z. Jia, O. Padon, J. Thomas, T. Warszawski, M. Zaharia, and A. Aiken, "TASO: Optimizing deep learning computation with automatic generation of graph substitutions," in *Proc. ACM SOSP*, 2019.

[27] H. Wang *et al.*, "PET: Optimizing tensor programs with partially equivalent transformations and automated corrections," in *Proc. USENIX OSDI*, 2021.

[28] J. Liu, J. Lin, F. Ruffy, C. Tan, J. Li, A. Panda, and L. Zhang, "NNSmith: Generating diverse and valid test cases for deep learning compilers," in *Proc. ACM ASPLOS*, 2023.

[29] F. Jia *et al.*, "CoDL: Efficient CPU-GPU co-execution for deep learning inference on mobile devices," in *Proc. ACM MobiSys*, 2022.

[30] J. S. Jeong *et al.*, "Band: Coordinated multi-DNN inference on heterogeneous mobile processors," in *Proc. ACM MobiSys*, 2022.

[31] A. Ignatov *et al.*, "AI benchmark: All about deep learning on smartphones in 2019," in *Proc. ICCV Workshops*, 2019.

[32] M. Almeida, S. Laskaridis, A. Mehrotra, L. Dudziak, I. Leontiadis, and N. D. Lane, "Smart at what cost? Characterising mobile deep neural networks in the wild," in *Proc. ACM IMC*, 2021.

[33] V. J. Reddi *et al.*, "MLPerf inference benchmark," in *Proc. ISCA*, 2020.

[34] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. SC*, 2015.

[35] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proc. ACM ASPLOS*, 2009.

[36] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable DETR: Deformable transformers for end-to-end object detection," in *Proc. ICLR*, 2021.

[37] S. Wellek, *Testing Statistical Hypotheses of Equivalence and Noninferiority*, 2nd ed. Chapman & Hall/CRC, 2010.

[38] M. Oquab *et al.*, "DINOv2: Learning robust visual features without supervision," *Transactions on Machine Learning Research*, 2024.

[39] O. Russakovsky *et al.*, "ImageNet large scale visual recognition challenge," *International Journal of Computer Vision*, 2015.

[40] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. CVPR*, 2016.

[41] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. CVPR*, 2018.

[42] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. ICML*, 2019.

[43] B. Recht, R. Roelofs, L. Schmidt, and V. Shankar, "Do ImageNet classifiers generalize to ImageNet?" in *Proc. ICML*, 2019.